ChartNav — Workflow State Machine Build

Phase: strict encounter state machine + filtered listing
Repo: `thekidd2227/chartnav-platform` · Generated 2026-04-17

Contents

1. Executive summary
2. 01 — Current state
3. 02 — Workflow state machine
4. 03 — API endpoints
5. 04 — Data model
6. 05 — Build log
7. 06 — Known gaps + verification matrix
8. 07 — Auth, org scoping, RBAC
9. 08 — Test strategy
10. Diagram — System architecture
11. Diagram — Encounter status machine
12. Diagram — ER
13. Diagram — API / data flow

Executive summary

This document consolidates **all** ChartNav backend phases delivered to

date. It covers the workflow spine, the strict encounter state machine,

encounter filtering, and the dev auth + org scoping layer that just

landed.

Phase 1 — Workflow spine. Alembic migration `a1b2c3d4e5f6` introduced `encounters` and `workflow_events` tables with FKs and indexes. Read and write endpoints for both. Idempotent seed.

Phase 2 — State machine + filtering. `POST /encounters/{id}/status` enforces explicit forward + rework edges; everything else returns `400 invalid_transition` with an allowed-next-states listing. `GET /encounters` accepts `organization_id`, `location_id`, `status`, `provider_name` filters.

Phase 3 — Dev auth + org scoping. New `apps/api/app/auth.py` resolves the caller from `X-User-Email`, looks up the user, and attaches a `Caller` dataclass with authoritative `organization_id`. Every

encounter route is now org-scoped: cross-org reads return 404, cross-org body/query assertions return 403.

Phase 4 — RBAC + full scoping + tests (this phase). New `apps/api/app/authz.py` introduces `admin` / `clinician` / `reviewer` roles and per-edge transition rules. `/organizations`, `/locations`, `/users` are now authenticated and org-scoped. The auth module gained a `CHARTNAV_AUTH_MODE` seam and a standardized `{error_code, reason}` error envelope. A real pytest suite (25 tests, TestClient + per-test SQLite) locks in auth, scoping, RBAC, state-machine invariants, and cross-org denial.

Preserved untouched: `/health`, `/`, Alembic history, SQLite dev workflow, the state machine + filtering surface, and the workflow_events model.

01 — Current state

ChartNav — Current State

As of: 2026-04-17 (phase: RBAC + full scoping + tests)

Repo layout (relevant)

```
chartnav-platform/
├── apps/
│   ├── api/
│   │   ├── app/main.py
│   │   ├── app/auth.py              # authn only (header transport today)
│   │   ├── app/authz.py             # NEW — RBAC roles + dependencies
│   │   ├── app/api/routes.py        # all routes authed + scoped + role-gated
│   │   ├── alembic.ini
│   │   ├── alembic/env.py           # now honors `-x sqlalchemy.url=` for tests
│   │   ├── alembic/versions/        # 2 migrations (unchanged this phase)
│   │   ├── scripts_seed.py          # 2 orgs, 5 users across 3 roles
│   │   ├── tests/                   # NEW — pytest + TestClient
│   │   │   ├── conftest.py
│   │   │   ├── test_auth.py
│   │   │   ├── test_scoping.py
│   │   │   └── test_rbac.py
│   │   └── pyproject.toml
│   └── web/
├── infra/docker/docker-compose.yml
└── docs/
    ├── build/                       # living docs
    ├── diagrams/                    # Mermaid sources
    └── final/                       # consolidated HTML/PDF
```

Runtime baseline

- Python 3.11+, FastAPI, raw `sqlite3` driver.
- SQLite at `apps/api/chartnav.db` (gitignored).
- Alembic head: `a1b2c3d4e5f6` (no new migrations this phase).
- Auth transport: `X-User-Email` → `users` lookup (dev only).
- Auth seam: `app.auth.require_caller` with `CHARTNAV_AUTH_MODE` env flag for future JWT/SSO swap.

- Authorization: `app.authz` with `ROLE_ADMIN` / `ROLE_CLINICIAN` / `ROLE_REVIEWER`, role-gated dependencies for create/event/transition.
- Error envelope standardized: `{"detail": {"error_code": "...", "reason": "..."}}`.

Verified working endpoints

Open

- `GET /health`, `GET /`

Authenticated (require `X-User-Email`)

- `GET /me`
- `GET /organizations` — scoped to caller's org (single row)
- `GET /locations` — scoped to caller's org
- `GET /users` — scoped to caller's org
- `GET /encounters` — scoped + filterable
- `GET /encounters/{id}` — 404 if cross-org
- `GET /encounters/{id}/events` — 404 if cross-org

Authenticated + role-gated

- `POST /encounters` — admin or clinician (RBAC); body `organization_id` must match caller
- `POST /encounters/{id}/events` — admin or clinician
- `POST /encounters/{id}/status` — per-edge RBAC (see `07-auth-and-scoping.md`)

Seeded tenants / users

org_id	slug	email	role
1	`demo-eye-clinic`	admin@chartnav.local	admin
1	`demo-eye-clinic`	clin@chartnav.local	clinician
1	`demo-eye-clinic`	rev@chartnav.local	reviewer
2	`northside-retina`	admin@northside.local	admin
2	`northside-retina`	clin@northside.local	clinician

02 — Workflow state machine

Encounter Workflow State Machine

Status values

Status	Meaning
`scheduled`	Visit is booked, patient has not arrived.
`in_progress`	Provider is seeing the patient / actively charting.
`draft_ready`	Note draft has been generated and is awaiting review.
`review_needed`	Reviewer flagged the note — needs revision or sign-off.
`completed`	Note signed off; encounter closed. **Terminal.**

Allowed transitions

Forward flow:

```
scheduled → in_progress → draft_ready → review_needed → completed
```

Explicit rework transitions (documented + enforced):

From	To	Purpose
`draft_ready`	`in_progress`	Draft rejected, return to charting.
`review_needed`	`draft_ready`	Reviewer kicks note back for rewrite.

Rejected transitions (examples that return HTTP 400)

- `scheduled → completed`
- `scheduled → review_needed`
- `scheduled → draft_ready`
- `in_progress → scheduled`
- `in_progress → completed` (must pass through draft_ready + review_needed)

- `completed → anything` (terminal)
- Any unknown status string (`invalid_status`)

Error body format:

```
{"detail":"invalid_transition: in_progress -> completed is not permitted; allowed next s
```

Idempotency

Posting the **same** status the encounter is already in is a no-op: the row

is returned unchanged and **no** `status_changed` event is appended.

Side effects on successful transition

Transition target	Timestamp action
`in_progress`	set `started_at = now()` if currently NULL
`completed`	set `completed_at = now()` ; also set `started_at = now()` if it was NULL
other	no timestamp mutation

Every successful transition appends a `workflow_events` row:

```
{
  "event_type": "status_changed",
  "event_data": {"old_status": "...", "new_status": "..."}
}
```

Diagram

See `docs/diagrams/encounter-status-machine.md` (Mermaid).

Source of truth

`apps/api/app/api/routes.py` :

- `ALLOWED_STATUSES` — the 5 permitted status strings.

- `ALLOWED_TRANSITIONS` — `dict[str, set[str]]`, one entry per source state.
- `update_encounter_status()` — the single enforcement point.

API Endpoints

Base URL (local dev): `http://127.0.0.1:8000`.

Auth: every endpoint tagged 🔒 requires header `X-User-Email: <seeded user email>`.

Some endpoints are further role-gated — see `07-auth-and-scoping.md`.

All error bodies have the standardized shape:

```
{"detail": {"error_code": "<stable_code>", "reason": "<human message>"}}
```

System (open)

Method	Path
GET	`/health`
GET	`/`

Identity

Method	Path	Auth	Roles
GET	`/me`	🔒	any

Org metadata (🔒, org-scoped, any role)

Method	Path	Behavior
GET	`/organizations`	Returns only the caller's org row.
GET	`/locations`	`WHERE organization_id = caller.org`.
GET	`/users`	`WHERE organization_id = caller.org`.

Encounters (🔒, org-scoped + RBAC)

GET `/encounters`

Any authenticated role. Always filtered to `caller.organization_id`.

Optional query params (AND-ed, parameterized):

Param	Rule
`organization_id`	Must equal `caller.org` — else 403.
`location_id`	Narrows within caller org.
`status`	Must be in `ALLOWED_STATUSES` — else 400.
`provider_name`	Exact match.

GET `/encounters/{id}` — any role

404 if missing or cross-org.

GET `/encounters/{id}/events` — any role

404 if parent missing or cross-org.

POST `/encounters` — admin, clinician

Body:

```
{
  "organization_id": 1,
  "location_id": 1,
  "patient_identifier": "PT-1003",
  "patient_name": "Alex Chen",
  "provider_name": "Dr. Ortiz",
  "scheduled_at": "2026-04-18T14:30:00Z",
  "status": "scheduled"
}
```

Rules:

- Reviewer → 403 `role_cannot_create_encounter`.
- `organization_id` must match caller org → 403 otherwise.
- `location_id` must exist and belong to caller org → 400 or 403 otherwise.
- Initial status restricted to `scheduled` or `in_progress`.

POST `/encounters/{id}/events` — admin, clinician

Reviewer → 403 `role_cannot_create_event`. 404 if cross-org.

POST `/encounters/{id}/status`

404 if cross-org. Status must be valid. Transition must be legal by the

state machine (`02-workflow-state-machine.md`). **Per-edge** RBAC:

From	To	Roles allowed
scheduled	in_progress	admin, clinician
in_progress	draft_ready	admin, clinician
draft_ready	in_progress	admin, clinician
draft_ready	review_needed	admin, reviewer
review_needed	draft_ready	admin, reviewer
review_needed	completed	admin, reviewer

Same-state POST = no-op (200). Role violation → 403 `role_cannot_transition`. Invalid transition → 400 `invalid_transition`.

Error code inventory

Code	HTTP	Origin
`missing_auth_header`	401	auth transport
`unknown_user`	401	auth transport
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to unknown
`role_forbidden`	403	generic `require_roles` gate
`role_cannot_create_encounter`	403	POST /encounters
`role_cannot_create_event`	403	POST /encounters/{id}/events
`role_cannot_transition`	403	POST /encounters/{id}/status
`cross_org_access_forbidden`	403	body/query asserts another org
`encounter_not_found`	404	missing or cross-org encounter
`location_not_found`	400	bad location_id

Code	HTTP	Origin
`invalid_status`	400	unknown status string
`invalid_initial_status`	400	POST /encounters with bad init
`invalid_transition`	400	disallowed state-machine edge

Data Model

SQLite (local dev). Schema produced by Alembic migrations

`43ccbf363a8f → a1b2c3d4e5f6`. No schema changes in this RBAC/scoping phase.

Tables (unchanged this phase)

`organizations`

column	type	constraints
id	INTEGER	PK
name	VARCHAR(255)	NOT NULL
slug	VARCHAR(255)	NOT NULL, UNIQUE
created_at	DATETIME	NOT NULL default now()

`locations`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
name	VARCHAR	NOT NULL
created_at	DATETIME	NOT NULL default now()

`users`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
email	VARCHAR	NOT NULL, UNIQUE
full_name	VARCHAR	NULL

column	type	constraints
role	VARCHAR	NOT NULL default `"admin"`
created_at	DATETIME	NOT NULL default now()

Application-level role vocabulary (enforced by `app/authz.py`):

`admin`, `clinician`, `reviewer`. The column remains a free VARCHAR in

SQL; constraint is enforced at the app layer. If you need a DB-level

CHECK constraint, add it in a future migration.

`encounters` and `workflow_events`

Unchanged.

Seeded tenants (two, with full role coverage)

Org 1 — `demo-eye-clinic` (id=1)

Location: `Main Clinic` (id=1).

email	role
admin@chartnav.local	admin
clin@chartnav.local	clinician
rev@chartnav.local	reviewer

Encounters:

id	patient	provider	status	events
1	PT-1001	Dr. Carter	`in_progress`	3
2	PT-1002	Dr. Patel	`review_needed`	5

Org 2 — `northside-retina` (id=2)

Location: `Northside HQ` (id=2).

email	role
admin@northside.local	admin

email	role
clin@northside.local	clinician

Encounters:

id	patient	provider	status	events
3	PT-2001	Dr. Ahmed	`scheduled`	1

Relationships

Unchanged — see `docs/diagrams/er-diagram.md`.

Event taxonomy

event_type	Writer	Shape
`encounter_created`	`POST /encounters` / seed	`{"status": "...", "created_by": "<email>"}`
`status_changed`	`POST /encounters/{id}/status`	`{"old_status": "...", "new_status": "...", "changed_by": "<email>"}`
user-supplied	`POST /encounters/{id}/events`	Any JSON

05 — Build log

Build Log

Reverse-chronological.

2026-04-17 — Phase: RBAC + full scoping + automated tests

Step 1 — Baseline

- Starting head: `efb5b56` (dev auth + org scoping).
- No schema changes needed this phase.

Step 2 — RBAC module

- New `apps/api/app/authz.py`:
- Role constants `ROLE_ADMIN`, `ROLE_CLINICIAN`, `ROLE_REVIEWER`.
- `CAN_CREATE_ENCOUNTER`, `CAN_CREATE_EVENT` sets.
- `TRANSITION_ROLES` map keyed on (from_status, to_status).
- Dependencies: `require_roles(*)`, `require_create_encounter`, `require_create_event`.
- `assert_can_transition(caller, from, to)` enforces per-edge RBAC with an admin safety-net for unmapped edges.

Step 3 — Routes wiring

- `POST /encounters` now depends on `require_create_encounter` → reviewer 403.
- `POST /encounters/{id}/events` now depends on `require_create_event` → reviewer 403.
- `POST /encounters/{id}/status` calls `assert_can_transition` after the state machine accepts the edge. Clinician cannot drive review-stage edges; reviewer cannot drive charting edges.
- `/organizations`, `/locations`, `/users` now require auth and filter `WHERE organization_id = caller.org` (previously unauthenticated).
- Error envelope standardized: every 4xx/5xx now returns `{detail: {error_code, reason}}`. Stable error codes inventoried in `03-api-endpoints.md`.

Step 4 — Auth seam hardening

- `apps/api/app/auth.py` now reads `CHARTNAV_AUTH_MODE` (default `"header"`) as the single swap point for future transports. Module docstring calls out dev vs. prod.
- Errors routed through a private `_error(code, reason, status)` helper so the shape is consistent with authz.
- Authn vs. authz strictly separated: `auth.py` answers "who" only, `authz.py` answers "may".

Step 5 — Seed expansion

- `scripts_seed.py` now provisions 5 users across 2 orgs covering all three roles (admin/clinician/reviewer in org1, admin/clinician in org2). Users upserted on re-seed so role changes in the seed converge deterministically. Still idempotent.

Step 6 — Automated tests

- New `apps/api/tests/` with pytest + FastAPI `TestClient`.
- Per-test temp SQLite fixture: each test gets a fresh migrated + seeded DB, so writes never leak across tests.
- Added `-x sqlalchemy.url=...` support in `alembic/env.py` so the test fixture can migrate an ephemeral DB without rewriting `alembic.ini`.
- Added `pytest` and `httpx` as dev deps.
- Tests cover: auth surface, org scoping across every list/read, cross-org 404, cross-org 403 lens, per-role create/event/transition rules, invalid transitions, event provenance (`changed_by`).
- Run: `pytest tests/ -v` → **25 passed** in ~12s.

Step 7 — Verification

- `alembic upgrade head` against a fresh DB → both migrations apply.
- Seed idempotent across back-to-back runs (user count stable).
- `pytest` suite green (25/25).
- Full manual sanity: `/me` across all seeded roles returns correct `role` + `organization_id`.

Step 8 — Documentation

- Updated `01-current-state.md`, `03-api-endpoints.md` (error inventory), `04-data-model.md` (role coverage), `05-build-log.md`, `06-known-gaps.md`, `07-auth-and-scoping.md` (RBAC section + diagram).
- New `08-test-strategy.md` covering harness, matrix, and run command.
- Regenerated `docs/final/chartnav-workflow-state-machine-build.{html,pdf}` with this phase's evidence.

Step 9 — Git hygiene

- DB reset to seeded state before commit.

- Gitignore already covers `.venv`, `__pycache__`, `.pytest_cache`, `*.db`.

Prior phases (preserved)

2026-04-17 — Dev auth + org scoping (phase 3)

Auth dependency, `X-User-Email` transport, `Caller` ctx, cross-org 404/403, two-tenant seed.

2026-04-17 — Strict state machine + filtering (phase 2)

`ALLOWED_TRANSITIONS` map, standardized 400 errors on invalid edges, `GET /encounters` filters.

2026-04-17 — Workflow spine (phase 1)

Alembic migration `a1b2c3d4e5f6`, idempotent seed, six encounter endpoints.

06 — Known gaps + verification matrix

Known Gaps & Verification Matrix

Automated verification (this phase)

```
$ cd apps/api && pytest tests/ -v
... 25 passed in 12.39s
```

Every matrix below is now machine-asserted in `apps/api/tests/`.

Auth (5/5)

Case	Expected	Result
`/health` open	200	✅
`/me` no header	401 `missing_auth_header`	✅
`/me` empty header	401	✅
`/me` unknown email	401 `unknown_user`	✅
`/me` admin org1	200 role=admin, org=1	✅

Scoping (8/8)

Case	Expected	Result
`/organizations` scoped per tenant	1 row each	✅
`/organizations` no auth	401	✅
`/locations` scoped	caller org only	✅
`/users` scoped	caller org only	✅
`/encounters` disjoint per tenant	no overlap	✅
Cross-org `GET /encounters/{id}`	404	✅
`?organization_id=<other>` lens	403	✅
Filter within org (`?status=`)	no leakage	✅

RBAC (12/12)

Case	Expected	Result
Admin creates encounter	201	✅
Clinician creates encounter	201	✅
Reviewer creates encounter	403 `role_cannot_create_encounter`	✅
Reviewer adds event	403 `role_cannot_create_event`	✅
Clinician: in_progress→draft_ready	200	✅
Clinician: review_needed→completed	403 `role_cannot_transition`	✅
Reviewer: review_needed→completed	200	✅
Reviewer: review_needed→draft_ready (kick back)	200	✅
Cross-org mutate	404	✅
Invalid transition (admin: in_progress→completed)	400 `invalid_transition`	✅
`status_changed` event written with `changed_by`	event count +1	✅
Cross-org create body mismatch	403 `cross_org_access_forbidden`	✅

Real gaps (prioritized for next phase)

1. **Transport still dev-only.** `X-User-Email` is spoofable. Swap to a signed token path via `CHARTNAV_AUTH_MODE`. The seam is in place; the dependency only needs a new branch.

2. **No user/org metadata writes.** `/organizations`, `/locations`, `/users` are read-only. Admins can't invite users, create locations, or edit their own org metadata through the API. Needs RBAC-gated write endpoints.

3. **Role constraint is app-layer only.** `users.role` is a free VARCHAR. Next migration could add a CHECK constraint or a lookup table.

4. **No pagination / cursor** on `GET /encounters`.

5. **No encounter update / delete / cancel** path; no `cancelled` status.

6. **Free-form `event_data`**. No per-`event_type` schema validation.

7. **Raw `sqlite3`** per-request connections. Postgres parity (docker compose) untested.

8. **CORS `allow_origins=["*"]` ** — tighten before any hosted deploy.

9. **No distinct audit log.** Auth failures and scoping violations are not persisted; only successful workflow events are.

10. **No rate limiting, no lockout, no observability.**

11. **No CI.** Tests exist but aren't yet run on every PR.

Auth, Org Scoping, RBAC

1. Authentication (authn)

Source: `apps/api/app/auth.py`.

- Transport today: request header `X-User-Email: <email>`.
- `require_caller` is a FastAPI dependency that resolves the user from

the `users` table and returns a `Caller(user_id, email, full_name, role, organization_id)`.

- Environment flag `CHARTNAV_AUTH_MODE` (default `"header"`) is the

production upgrade seam. When JWT/SSO lands, only the body of

`require_caller` (or a new branch in it) changes. Every route and

every RBAC helper continues to work unchanged because they depend on

`Caller`, not on any header.

Dev-only, explicitly

`X-User-Email` is trivially spoofable. This is acceptable for local

development only. **Do not deploy this to a shared environment

without first swapping the transport.** The seam is there; use it.

Standardized error envelope

Every auth/authz error returns:

```
{"detail": {"error_code": "<stable_code>", "reason": "<human message>"}}
```

Authn error codes

Code	HTTP	When
`missing_auth_header`	401	`X-User-Email` absent or empty.

Code	HTTP	When
`unknown_user`	401	Email not found in `users`.
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to something unwired.

2. Authorization (authz / RBAC)

Source: `apps/api/app/authz.py`.

Roles

Role	Intent
`admin`	Full read/write inside own org.
`clinician`	Charting side of the workflow.
`reviewer`	Review side of the workflow; read-only on create/events.

All seeded users today carry exactly one of these roles.

Permission surface

Surface	admin	clinician	reviewer
Read org / locations / users	✓	✓	✓
List / read encounters	✓	✓	✓
Read encounter events	✓	✓	✓
Create encounter	✓	✓	×
Add workflow event	✓	✓	×
Transition scheduled→in_progress	✓	✓	×
Transition in_progress→draft_ready	✓	✓	×
Transition draft_ready→in_progress (rework)	✓	✓	×
Transition draft_ready→review_needed	✓	×	✓
Transition review_needed→draft_ready (kick back)	✓	×	✓
Transition review_needed→completed	✓	×	✓

Authz dependencies

- `require_roles(*roles)` — generic gate, returns `Caller`.
- `require_create_encounter` — applied to `POST /encounters`.
- `require_create_event` — applied to `POST /encounters/{id}/events`.
- `assert_can_transition(caller, from, to)` — called by the status

handler **after** the state machine accepts the edge.

Authz error codes

Code	HTTP	When
`role_forbidden`	403	Generic `require_roles` denies.
`role_cannot_create_encounter`	403	Non-admin/clinician POSTs `/encounters`.
`role_cannot_create_event`	403	Non-admin/clinician POSTs event.
`role_cannot_transition`	403	Role may not drive that specific state edge.

3. Org scoping

Caller's `organization_id` is **authoritative**. Bodies and query

strings cannot override it.

Route	Scoping
`GET /organizations`	`WHERE id = caller.org`
`GET /locations`	`WHERE organization_id = caller.org`
`GET /users`	`WHERE organization_id = caller.org`
`GET /encounters`	`WHERE organization_id = caller.org` + filters
`GET /encounters/{id}`	404 if row is cross-org
`GET /encounters/{id}/events`	404 if parent cross-org
`POST /encounters`	Body `organization_id` must equal caller's; location must belong
`POST /encounters/{id}/events`	404 if cross-org
`POST /encounters/{id}/status`	404 if cross-org

404 vs 403

- **404** when the target might-or-might-not exist in another org —

returning 403 there would leak existence.

- **403** when the client explicitly asserts a different org (body or

`?organization_id=`) — the intent is unambiguous, fail loudly.

4. Diagram

5. What this phase explicitly does NOT do

- No password auth, JWT, or SSO integration.
- No scoped writes to organizations / locations / users (meta admin).
- No audit log distinct from `workflow_events` .
- No rate limiting.
- No per-tenant key management.

These are the next auth/security phase's targets.

Test Strategy

Stack

- `pytest` + FastAPI `TestClient` (httpx under the hood).
- Per-test temporary SQLite file: each test gets a fresh migrated +

seeded DB so writes from one test cannot leak into another.

- No external services; tests run offline in under ~15 seconds.

Layout

```
apps/api/tests/
├── conftest.py          # fixtures: test_db, client, seeded_ids, role headers
├── test_auth.py         # authn surface
├── test_scoping.py      # org scoping across every list/read endpoint
└── test_rbac.py         # role-gated writes + state transitions
```

Coverage matrix

Auth (`test_auth.py`)

- `/health` open.
- `/me` without header → 401 `missing_auth_header`.
- `/me` with empty header → 401.
- `/me` with unknown email → 401 `unknown_user`.
- `/me` with seeded admin → 200 + correct `organization_id` + role.

Scoping (`test_scoping.py`)

- `/organizations` returns only caller org row (for both orgs).
- `/organizations` without auth → 401.
- `/locations` scoped to caller org.

- `/users` scoped to caller org (no cross-tenant leakage).
- `/encounters` returns disjoint sets for org1 vs org2.
- Cross-org `GET /encounters/{id}` → 404.
- `?organization_id=2` as org1 caller → 403.
- Filters (`status=`) work inside caller org and don't leak.

RBAC (`test_rbac.py`)

- Admin can create encounter (201).
- Clinician can create encounter (201).
- Reviewer cannot create encounter (403 `role_cannot_create_encounter`).
- Reviewer cannot add events (403 `role_cannot_create_event`).
- Clinician can perform `in_progress → draft_ready` .
- Clinician cannot perform `review_needed → completed` (403 `role_cannot_transition`).
- Reviewer can complete a review (`review_needed → completed`).
- Reviewer can kick back (`review_needed → draft_ready`).
- Cross-org mutate → 404 (existence not leaked).
- Invalid transition still → 400 `invalid_transition` .
- Successful status change writes a `status_changed` event with `old_status` / `new_status` / `changed_by` .
- Cross-org body mismatch on `POST /encounters` → 403 `cross_org_access_forbidden` .

How to run

From `apps/api/` with the venv active:

```
pytest tests/ -v
```

Result at time of writing: **25 passed in ~12s**.

Guarantees we get from this suite

- Any regression that breaks auth, scoping, or a role gate surfaces in CI (when we add it).
- The error-code envelope is asserted against, so rename-by-accident is caught.
- Seed idempotency isn't tested directly here but is still exercised

indirectly: each test reseeds cleanly without collision.

Gaps not yet covered

- Concurrent writers / race conditions.
- Event JSON schema per `event_type`.
- Pagination behavior (not yet implemented).
- Auth transport swap (would warrant its own harness).

Diagram — System architecture

System Architecture

```
flowchart LR Client["Client
(curl / tests / apps/web)"] FastAPI["FastAPI
apps/api/app/main.py"] Authn["require_caller
apps/api/app/auth.py
(AUTH_MODE seam)"] Authz["require_roles / assert_can_transition
apps/api/app/authz.py"] Router["APIRouter
apps/api/app/api/routes.py"] SM["State Machine
ALLOWED_TRANSITIONS"] Scope["Org Scoping
ensure_same_org / 404 on cross-org"] DB[("SQLite
apps/api/chartnav.db")] Alembic["Alembic
apps/api/alembic/"] Seed["scripts_seed.py
2 orgs · 5 users · 3 roles"] Tests["pytest
apps/api/tests/"] Client -->|"HTTP + X-User-Email"| FastAPI FastAPI --> Authn Authn -->|"lookup by
email"| DB Authn -->|"Caller(role, org)"| Authz Authz --> Router Router --> Scope Router --> SM
Router -->|"scoped SQL"| DB Alembic -.->|"upgrade head"| DB Seed -.->|"idempotent seed"| DB
Tests -.->|"TestClient + temp DB"| FastAPI
```

Diagram — Encounter status machine

Encounter Status State Machine

stateDiagram-v2 [*] --> scheduled [*] --> in_progress : walk-in scheduled --> in_progress in_progress --> draft_ready draft_ready --> review_needed draft_ready --> in_progress : rework review_needed --> completed review_needed --> draft_ready : kick back completed --> [*]

Legend: any edge not shown is rejected at `POST /encounters/{id}/status`

with HTTP 400 `invalid_transition`.

Diagram — ER

ER Diagram

erDiagram organizations ||--o{ locations : "has" organizations ||--o{ users : "employs (identity + role)" organizations ||--o{ encounters : "owns" locations ||--o{ encounters : "hosts" encounters ||--o{ workflow_events : "emits" organizations { int id PK string name string slug UK datetime created_at } locations { int id PK int organization_id FK string name datetime created_at } users { int id PK int organization_id FK string email UK string full_name string role "admin | clinician | reviewer" datetime created_at } encounters { int id PK int organization_id FK int location_id FK string patient_identifier string patient_name string provider_name string status datetime scheduled_at datetime started_at datetime completed_at datetime created_at } workflow_events { int id PK int encounter_id FK string event_type text event_data datetime created_at }

Seeded tenants & users

org_id	slug	email	role
1	`demo-eye-clinic`	admin@chartnav.local	admin
1	`demo-eye-clinic`	clin@chartnav.local	clinician
1	`demo-eye-clinic`	rev@chartnav.local	reviewer
2	`northside-retina`	admin@northside.local	admin
2	`northside-retina`	clin@northside.local	clinician

`users.email` is the authentication key consumed from `X-User-Email`.

`users.role` is the RBAC key consumed by `app.authz`.

`users.organization_id` is the authoritative source of scope; never

derived from client input.

Diagram — API / data flow

API / Data Flow

sequenceDiagram autonumber participant C as Client participant Authn as require_caller participant Authz as authz gate participant R as Route handler participant SM as State Machine participant DB as SQLite Note over C,DB: Happy path — clinician advances own encounter C->>Authn: POST /encounters/1/status {"status":"draft_ready"}
X-User-Email: clin@chartnav.local Authn->>DB: SELECT user WHERE email=? DB-->>Authn: {role=clinician, organization_id=1} Authn-->>R: Caller(role=clinician, org=1) R->>DB: SELECT encounter 1 DB-->>R: row(org=1, status=in_progress) R->>R: encounter.org == caller.org ✓ R->>SM: is (in_progress → draft_ready) allowed? SM-->>R: yes R->>Authz: assert_can_transition(clinician, in_progress, draft_ready) Authz-->>R: allowed R->>DB: UPDATE + INSERT status_changed{changed_by=clin@...} R-->>C: 200 updated Note over C,DB: Role denied — clinician tries review-stage edge C->>Authn: POST /encounters/2/status {"status":"completed"}
X-User-Email: clin@chartnav.local Authn-->>R: Caller(role=clinician, org=1) R->>DB: SELECT encounter 2 DB-->>R: row(org=1, status=review_needed) R->>SM: is (review_needed → completed) allowed? SM-->>R: yes R->>Authz: assert_can_transition(clinician, review_needed, completed) Authz-->>C: 403 role_cannot_transition Note over C,DB: Reviewer cannot create encounter C->>Authn: POST /encounters {...}
X-User-Email: rev@chartnav.local Authn-->>Authz: Caller(role=reviewer) Authz-->>C: 403 role_cannot_create_encounter Note over C,DB: Cross-org read — returns 404 C->>Authn: GET /encounters/3
X-User-Email: clin@chartnav.local Authn-->>R: Caller(org=1) R->>DB: SELECT encounter 3 DB-->>R: row(org=2) R-->>C: 404 encounter_not_found Note over C,DB: Missing / unknown identity C->>Authn: GET /encounters Authn-->>C: 401 missing_auth_header C->>Authn: GET /encounters
X-User-Email: ghost@nowhere Authn->>DB: SELECT user DB-->>Authn: ∅ Authn-->>C: 401 unknown_user